

Bridge Structural Health Monitoring via a Machine Learning-Based Anomaly Detection Method

Internship Report – CEMOSA, Málaga (Proyect IAM4RAIL)

Trainee: Pedro Ignacio Sáez Díaz
Supervisors: Jose Solís Hernández
Israel Alejandro Hernández González

July 2025



1 Introduction

This report documents my participation in the structural instrumentation and monitoring project of the Arroyo de las Huertas de Mateo viaduct, located in Minglanilla (Cuenca, Spain), carried out during my internship period in the R&D department in CEMOSA. The viaduct, a critical structure for the Madrid-Valencia high-speed rail line (PK 202+097), is the subject of an extensive study led by the Central Laboratory of Structures and Materials (LCEYM) of CEDEX, in collaboration with ADIF and the Europe's Rail Joint Undertaking consortium (EU-Rail JU)[1].

The main objective of the project is to monitor in real time the structural behavior of the deck and its supports, analyzing two key phenomena:

- Long-term deformations (seasonal thermal effects, concrete rheology).
- Dynamic response to passing trains, including accelerations and displacements.

To achieve this, an advanced sensor system was deployed (LVDTs, triaxial accelerometers, inclinometers) along with distributed data acquisition units across 5 nodes (E1, P1, P5, P9 and P15) along the 995-meter viaduct (Figure 1). The instrumentation, operating under EtherCAT standards and synchronized via GPS, captures variables such as relative deck-to-substructure displacements ($\pm 300\text{mm}$), temperatures (-50°C to $+150^\circ\text{C}$), pile inclinations ($\pm 5^\circ$), and accelerations ($\pm 2g$).

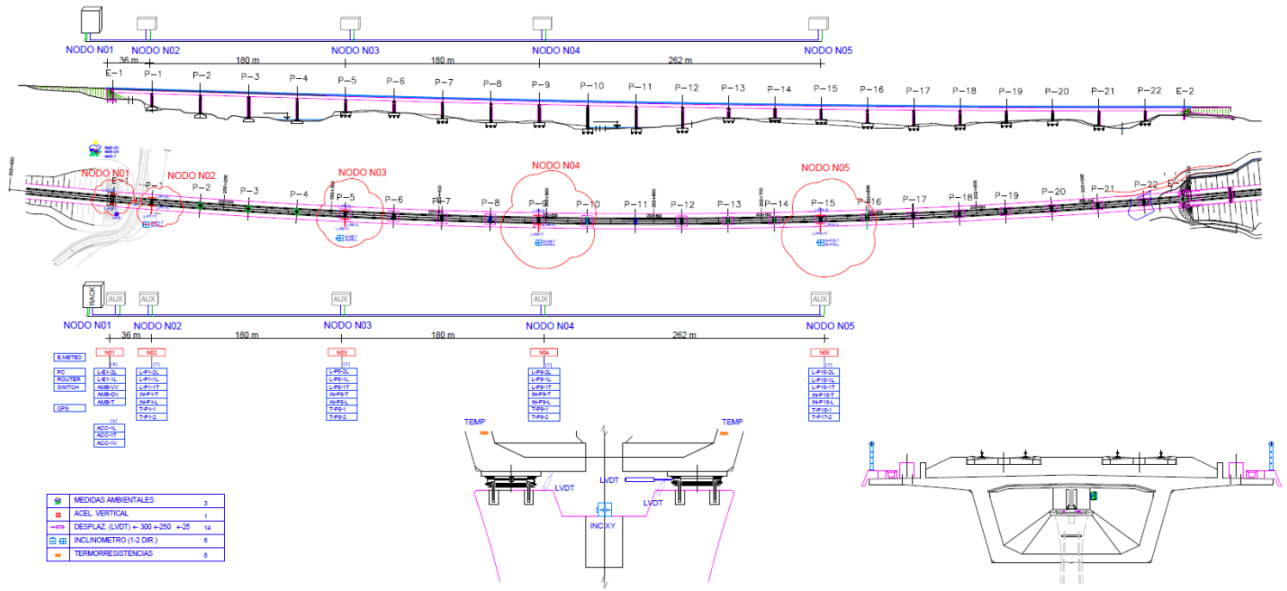


Figure 1: Data acquisition units along viaduct, localization of sensors.

My role focused on the design of a machine learning-based anomaly detection framework for maintenance decisions. The first step was to model the structural dynamics using a Kalman filter to generate residuals that were analyzed through power spectral density and a Bayesian whiteness test, with the goal of detecting damage specific correlations to quantify potential damage. Then, to account for environmental influences in the sensor signals a two-layer stacking ensemble was implemented to remove systematic trends. The cleaned data was then transformed into 2D tensors via the Fast Fourier Transform (FFT), enabling the application of a TimesBlock deep learning architecture. This model extracted periodic features and reconstruction errors, which were analyzed using Extreme Value Analysis (EVA) with a Generalized Pareto Distribution (GPD) to detect anomalies. To validate the model, synthetic noise-based anomalies were injected into the signal. Performance of the model was assessed using ROC curves and F1 scores. This report describes these activities, the technical challenges encountered, and the lessons learned within the framework of applied research on structural health monitoring.

Section 2 describes the Kalman filtering and power spectral density analysis, Section 3 covers data pre-processing and temperature drift compensation, Section 4 explains the anomaly detection via a TimesBlock module, and Section 5 presents model validation using synthetic anomalies.

2 Kalman Filtering and Power Spectral Density

This section is based on the methodology and theoretical framework presented in [2]. The derivations and conceptual explanations largely follow the original work but are adapted within the context of this project’s structural health monitoring problem.

2.1 Kalman Filtering: Motivation

A significant challenge in Structural Health Monitoring (SHM) lies in the fact that variations in the dynamic behavior of a structure do not necessarily imply structural damage. Environmental conditions, and most notably temperature, can cause substantial fluctuations in dynamic properties that may mask or mimic damage, thereby reducing the reliability of damage detection algorithms.

Multiple studies ([2], [3]) have shown that temperature-induced changes can produce global variations across the entire structure, often blocking the effects of actual physical deterioration. This overlap complicates the task of distinguishing between benign environmental influences and true structural faults. As a result, SHM techniques must incorporate strategies to disentangle environmental variability from damage-specific changes.

This section introduces a potential candidate that aims to separate structural damage from temperature effects by using **Kalman filtering**. In this approach, a Kalman filter is used to generate residuals, defined as the discrepancy between measured and estimated structural responses. Under ideal filtering conditions, these residuals should exhibit white-noise behavior, meaning their power spectral density (PSD) is flat across frequencies.

Two critical properties of the residual PSD are exploited:

1. **Environmental effects** induce global and relatively uniform shifts in the PSD, maintaining its flatness and preserving whiteness.
2. **Structural damage** leads to localized perturbations in the PSD, introducing peaks that result in a correlated stochastic process.

Thus, the emergence of spectral peaks or correlated behavior in the residuals serves as an indicative of damage, while temperature-induced variations leave the residuals largely uncorrelated. To statistically assess whether a given residual sequence qualifies as white noise, a **Bayesian whiteness test** is applied. This test involves generating predicted residual sequences for the undamaged structure and comparing the number of sign changes to those of the observed residuals, under the assumption that a white residual indicates no structural damage. A damage index D is also introduced from the residual PSD; the scaled coefficient of variation of the spectral density. When the structure is healthy, the index D is close to zero, while it increases towards one as damage in the structure becomes more severe.

2.2 Theory

Structural systems subjected to slowly varying environmental conditions can often be approximated as linear time-invariant systems over short time intervals (on the order of minutes) [2]. The system’s state vector at time t , denoted $\mathbf{x}(t) \in \mathbb{R}^n$, includes the sensor’s variables, a bias and their corresponding velocities. Both the bias and the velocity components act as latent variables—they are not directly observed but are estimated within the Kalman filtering framework.

The time evolution of the state $\mathbf{x}(t)$ is modeled by the stochastic differential equation:

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}w(t). \quad (1)$$

Here, $\mathbf{A} \in \mathbb{R}^{n \times n}$ is the state transition matrix, and $\mathbf{B} \in \mathbb{R}^{n \times r}$ is the input (or disturbance) matrix. The term $w(t)$ represents a zero-mean white noise process with spectral density matrix $\mathbf{Q} \in \mathbb{R}^{r \times r}$, which models the stochastic influence of the environment. The output (measurement) vector $\mathbf{y}(t)$ encapsulates the real-time sensor measurements and is related to the state space $\mathbf{x}(t)$ by:

$$\mathbf{y}(t) = \mathbf{C}\mathbf{x}(t) + \mathbf{D}w(t) + \eta(t) = \mathbf{C}\mathbf{x}(t) + \nu(t), \quad (2)$$

where $\mathbf{C} \in \mathbb{R}^{m \times n}$ is the state to the output matrix and $\mathbf{D} \in \mathbb{R}^{m \times r}$ is the direct transmission matrix. The stochastic processes η (used to model measurement noise) and ν are white processes assumed to be zero mean and statistically independent. The process ν has spectral density $\mathbf{R}$.

When the matrices $\{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}, \mathbf{Q}, \mathbf{R}\}$ are known, an optimal estimation of the state can be computed. If the system is observable and controllable the filtering process reaches a unique steady-state solution:

$$\dot{\hat{\mathbf{x}}} = \mathbf{A}\hat{\mathbf{x}}(t) + \mathbf{K}e(t) = (\mathbf{A} - \mathbf{K}\mathbf{C})\hat{\mathbf{x}}(t) + \mathbf{K}y(t), \quad (3)$$

where $\hat{\mathbf{x}}$ is the estimate of $\mathbf{x}$, the matrix $\mathbf{K} \in \mathbb{R}^{n \times m}$ is the **Kalman gain** and $e(t)$ is the measurement residual, given by:

$$e(t) = y(t) - \mathbf{C}\hat{\mathbf{x}}(t). \quad (4)$$

The steady-state Kalman gain $\mathbf{K}$ is given by:

$$\mathbf{K} = \mathbf{P}\mathbf{C}^T\mathbf{R}^{-1}, \quad (5)$$

where $\mathbf{P} = \mathbb{E}[(\hat{\mathbf{x}} - \mathbf{x})(\hat{\mathbf{x}} - \mathbf{x})^T]$ is the state error covariance matrix, computed as the solution of the **Ricatti equation**:

$$\mathbf{A}\mathbf{P} + \mathbf{P}\mathbf{A}^T + \mathbf{B}\mathbf{Q}\mathbf{B}^T - \mathbf{P}\mathbf{C}^T\mathbf{R}^{-1}\mathbf{C}\mathbf{P} = 0. \quad (6)$$

The state estimation error can be defined as $\tilde{\mathbf{x}}(t) = \hat{\mathbf{x}}(t) - \mathbf{x}(t)$. In terms of $\tilde{\mathbf{x}}$ the residual becomes $e(t) = -\mathbf{C}\tilde{\mathbf{x}}(t) + \nu(t)$.

The **residual correlation matrix** $\mathcal{R}_{ee}(t_2, t_1)$ for $t_2 > t_1$ is then given by:

$$\mathcal{R}_{ee}(t_2, t_1) := \mathbb{E}[e(t_2)e(t_1)^T] = \mathbf{C}\mathbb{E}[\tilde{\mathbf{x}}(t_2)\tilde{\mathbf{x}}(t_1)]\mathbf{C}^T - \mathbf{C}\mathbb{E}[\tilde{\mathbf{x}}(t_2)\nu(t_1)^T] + \mathbf{R}\delta(t_2 - t_1). \quad (7)$$

Under the assumptions that the system operates under ideal conditions and stationarity, the Kalman filter is optimal, and the residual $e(t)$ behaves as a zero-mean, uncorrelated white stochastic process. The residual correlation matrix simplifies to:

$$\mathcal{R}_{ee}(t_2, t_1) = \mathbf{R}\delta(t_1 - t_2). \quad (8)$$

Moreover, for stationary processes the correlation matrix of the residual is related to the power spectral density (PSD) of the residual by the **Wiener-Khinchin theorem**:

$$\mathcal{R}_{ee}(\tau) = \int_{-\infty}^{\infty} \mathcal{S}_{ee}(\omega)e^{i\omega\tau}d\omega \Leftrightarrow \mathcal{S}_{ee}(\omega) = 2\pi\mathbf{R} \quad \forall \omega \in (-\infty, \infty). \quad (9)$$

This directly implies that both the power spectral density (PSD) of the residual and the residual correlation is constant. Therefore, under ideal conditions, the residual $e(t)$ is a white noise process in both time and frequency domains. Under smooth variations of the temperature field, or other environmental conditions, the residual spectral density remains approximately constant for an undamaged structure, but becomes correlated if structural damage occurs.

2.3 Bayesian Whiteness Test

From the previous section 2.2, when the Kalman filter operates under optimal conditions the stochastic process $e(t)$ is a white process. Correlations in the residuals are introduced by structural damage, while the residual remains unaffected by temperature variations. This property forms the basis for residual-based damage detection. Testing the residual $e(t)$ for statistical whiteness allows to determine whether the underlying structure has changed, as structural damage induces frequency-specific peaks in $\mathcal{S}_{ee}(\omega)$.

A test statistic WT is defined to measure the discrepancy between the model and the data. The test statistic proposed to measure the whiteness of the residual sequence consists in the number of sign changes of $e(t)$. If the residual is white the statistic WT has a large value due to the (erratic) nature of the residual, whereas WT decreases as the residual correlation increases since the time-history becomes smoother. The value of the statistic is compared to a Bayesian p -value, defined as the probability that the simulated data could be more extreme than the observed data:

$$p = \mathbb{P}(WT(e) \geq WT(\bar{e})|\mathbf{Y}) \quad (10)$$

The simulated or predicted sequence of the residual for the undamaged structure is denoted $\bar{e}$, and $\mathbf{Y}$ is the measured signal from the sensors.

2.4 Damage Quantification

The second feature for damage evaluation is a peak statistic, which tests for the presence of peaks in a given frequency bandwidth $[\omega_1, \omega_2]$. For such frequency-bandwidth the truncated spectral moments are defined as:

$$\lambda_j = \int_{\omega_1}^{\omega_2} \omega^j \mathcal{S}_{ee}(\omega) d\omega. \quad (11)$$

The proposed normalized coefficient D by [2] to quantify damage is given by:

$$D = 1 - \sqrt{3} \frac{\omega_2 + \omega_1}{\omega_2 - \omega_1} \frac{\lambda_0}{\lambda_1} \sqrt{\frac{\lambda_2}{\lambda_0} - \left(\frac{\lambda_1}{\lambda_0}\right)^2}. \quad (12)$$

This definition is such that when the system is healthy (constant spectral density), the coefficient D is close to zero, but as the damage increases, the index approaches one.

2.5 Implementation

This section outlines the implementation of the Kalman Filter, along with the two damage-sensitive features introduced earlier: the whiteness test (Section 2.3) and the scaled coefficient of variation of the spectral density, denoted as D (Section 2.4).

In this project the sensor data consists of measurements from five pot bearings. Each bearing provides six measurements: two temperature values (north T_N and south T_S), two angular inclinations (about the x -axis θ_x and the y -axis θ_y), and two displacements (fixed-end d_1 and transversal d_2). Therefore, the output measurement vector $\mathbf{y}(t)$ is defined as:

$$\mathbf{y}(t) = [T_N, T_S, \theta_x, \theta_y, d_1, d_2]^T. \quad (13)$$

The state vector $\mathbf{x}(t)$ contains the temperature measurements, a bias term, the angular inclinations and displacements, as well as their corresponding velocities. It is defined as:

$$\mathbf{x}(t) = [T_N, T_S, \text{bias}, \theta_x, v_{\theta_x}, \theta_y, v_{\theta_y}, d_1, v_{d_1}, d_2, v_{d_2}]^T. \quad (14)$$

The output matrix $\mathbf{C} \in \mathbb{R}^{6 \times 11}$ maps the observable states from $\mathbf{x}(t)$ to $\mathbf{y}(t)$. The state-transition matrix $\mathbf{A} \in \mathbb{R}^{11 \times 11}$ encodes the baseline (undamaged) dynamics of the viaduct structure. It acts as the reference model used by the Kalman Filter to detect deviations caused by damage. The matrix $\mathbf{A}$ is constructed to reflect the mechanical dynamics of the structure. It is derived from the mass, damping, and stiffness characteristics of the system. These dynamics can be approximated using modal decomposition. Although a true finite element model (FEM) would allow for full spatial discretization and physical accuracy, the proposed matrix $\mathbf{A}$ is an approximation of the structure as a multi-degree-of-freedom (MDOF) system with four dominant vibration modes. Temperature dependence is not explicitly modeled in the stiffness terms, but could be incorporated by scaling the modal frequencies according to temperature-dependent elastic moduli.

For each vibration mode, the governing equation is modeled as a damped second-order differential equation:

$$\ddot{q} + 2\zeta\omega\dot{q} + \omega^2 q = 0, \quad (15)$$

where ζ is the damping ratio, and ω is the natural frequency of the mode.

To construct the discrete-time state-transition matrix, the solution to this equation is evaluated at $t + \Delta t$, giving the transition matrix for each mode:

$$\mathbf{A}_{\text{mode}} = e^{-\zeta\omega\Delta t} \begin{bmatrix} \cos(\omega_d\Delta t) + \frac{\zeta\omega}{\omega_d} \sin(\omega_d\Delta t) & \frac{1}{\omega_d} \sin(\omega_d\Delta t) \\ -\omega^2 \sin(\omega_d\Delta t) & \cos(\omega_d\Delta t) - \frac{\zeta\omega}{\omega_d} \sin(\omega_d\Delta t) \end{bmatrix},$$

where $\omega_d = \omega\sqrt{1 - \zeta^2}$ is the damped natural frequency.

These modal matrices are then assembled into a block-diagonal structure to form the global $\mathbf{A}$ matrix that governs the evolution of the full system state. Future improvements could incorporate true FEM-based models and temperature-adaptive stiffness matrices.

For the current implementation, the Kalman filter state-transition matrix $\mathbf{A}$ adopts a generic form. The underlying dynamic model assumes constant velocity over discrete time steps and is defined as:

$$\begin{cases} q_{k+1} = q_k + dt \cdot \dot{q}_k \\ \dot{q}_{k+1} = \dot{q}_k \end{cases} \quad (16)$$

which results in the matrix $\mathbf{A}$ to be a block-matrix with blocks:

$$\mathbf{A}_{\text{block}} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}. \quad (17)$$

The matrix $\mathbf{C}$ is defined as a sparse matrix that selects the entries in the state $\mathbf{x}(t)$ that are directly measured by the sensors and are summarized in the output vector $\mathbf{y}(t)$. The discrete-time Kalman filter is implemented in Python following the formulation presented in [4], which includes both the prediction (time update) and correction (measurement update) steps.

2.6 Illustrative Results

The Kalman filter was successfully implemented using the approach described in [4]. Figure 2 presents the filter's application to time-series data collected from an inclinometer sensor. In Figure 3, the power spectral density (PSD) of the residuals is shown for multiple pot bearings.

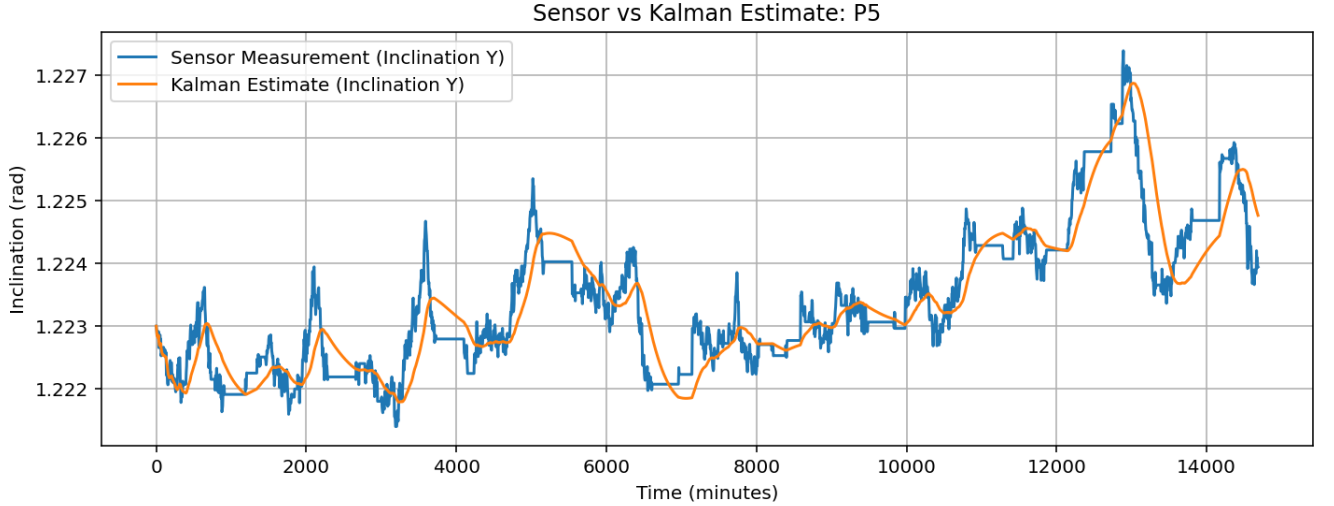


Figure 2: Kalman filter output for inclinometer data at pot bearing P5. The plot shows the measured signal alongside the estimated state, which illustrates the filter's ability to reduce noise in the signal from environmental effects.

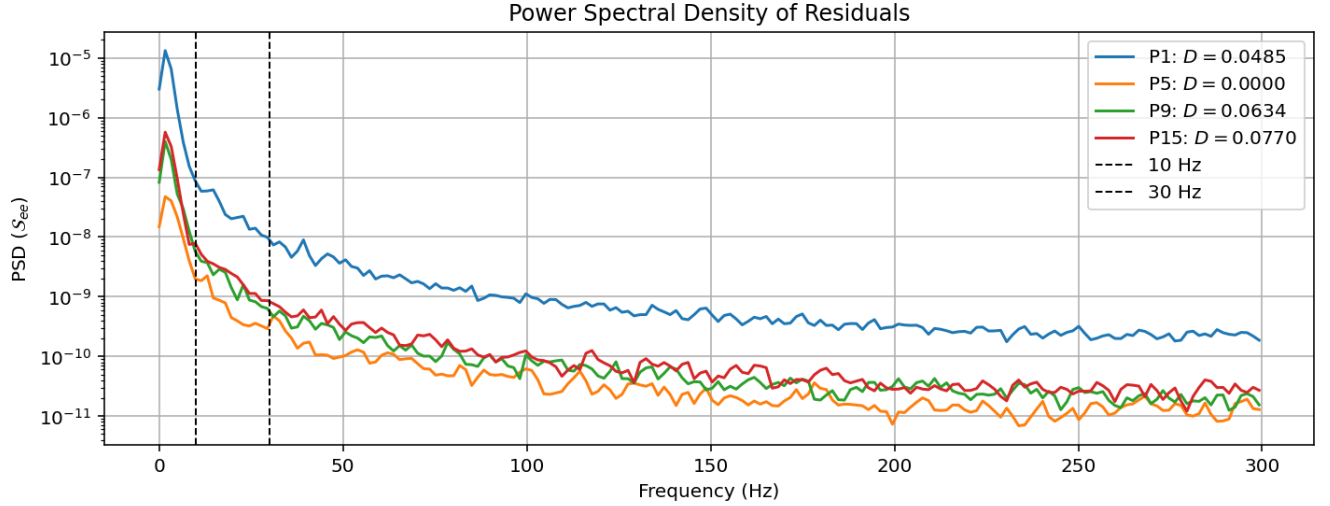


Figure 3: Power spectral density of residuals with normalized D for the frequency window [10 Hz, 30 Hz]

The attenuation observed in the low-frequency range is likely attributable to the oversimplified modeling of system dynamics—particularly the state transition matrix $\mathbf{A}$, the control input matrix $\mathbf{B}$, and the noise covariance matrix $\mathbf{Q}$. These matrices require further refinement to accurately reflect the physical behavior of the signals.

In particular, matrix $\mathbf{A}$ should be constructed based on a physically meaningful dynamic model. Instead of the basic approximation used (Equation 16) a more accurate formulation could involve deriving $\mathbf{A}$ from either a finite element method (FEM) or the proposed multi-degree-of-freedom (MDOF) system with four degrees of freedom. These should be configured using the bridge’s known natural frequencies, which would better capture its vibration characteristics.

To assess the statistical properties of the Kalman filter residuals, a Bayesian whiteness test was conducted. Assuming that white noise exhibits at least 40% sign changes relative to the residual length, the test revealed that **none** of the residuals satisfied the whiteness criterion. This implies that the residuals contain structured temporal patterns and violate the white noise assumption that is characteristic of an optimally tuned Kalman filter. While this behavior could suggest the presence of structural anomalies or damage (see Section 2.3), it is more likely a reflection of limitations in the current dynamic model and the parameter choices used within the Kalman filter. This interpretation is supported by the fact that a key assumption for the power spectral density to remain approximately constant is that the Kalman filter operates optimally. Future work could focus on refining the model and tuning filter parameters to improve the performance and reliability of the Bayesian whiteness test for damage detection.

3 Data Pre-Processing

This section describes the methodology for modeling temperature effects to enable drift compensation via a two-layer stacking ensemble algorithm, as detailed in [5]. This is a key step in data pre-processing in order to improve the accuracy of anomaly detection.

3.1 Data Cleaning and Time Alignment

The raw datasets—inclination, displacement (LVDT), and temperature—were first loaded and converted to structured time series. Each dataset included a Timestamp column, which was parsed and converted to date time format. To align the datasets to a common sampling frequency, all sensor readings were downsampled to 10-minute intervals (1/600 Hz) by computing the mean within each window. To preserve continuity, any missing values introduced during downsampling were filled using **forward fill** (`ffill`). Finally, a common time index was established by taking the intersection of timestamps across all three datasets to ensure full temporal alignment.

3.2 Two-Layer Stacking Ensemble

The steps described in this section closely follow the methodology presented in [5].

Motivation: SHM sensor data often exhibit long-term variations due to environmental factors such as temperature. These variations, sometimes referred to as **data drift**, are typically smooth and systematic rather than abrupt or irregular. While such changes may resemble anomalies, they are in fact predictable and physically explainable. Mitigating temperature-induced drift is crucial for reducing false positives in anomaly detection systems.

The data cleaning framework applied in this project involves the application of a **two-layer stacking ensemble** to reduce the data drift in the data previous to anomaly detection. The main idea behind the two-layer stacking ensemble is that the output of the first layer (consisting of various base models) is used as an input for the Meta-Model.

The original dataset is denoted as: $S = \{(x_1, y_1), \dots, (x_T, y_T)\}$, where each pair (x_i, y_i) consists of input features x_i and a corresponding target values y_i at each time instant. The vector $x_i \in \mathbb{R}^2$ contains explanatory variables (in this case two values of temperature), while $y_i \in \mathbb{R}^C$ represents the sensor response to be corrected, such as displacement, inclination, or other physical quantities derived from the monitoring system. Here, C represents the number of sensor response variables, which in this study is equal to $C = 5$ (2 values for inclination (x and y) and three values for displacement (LVDT sensor readings for transversal, fixed and free)).

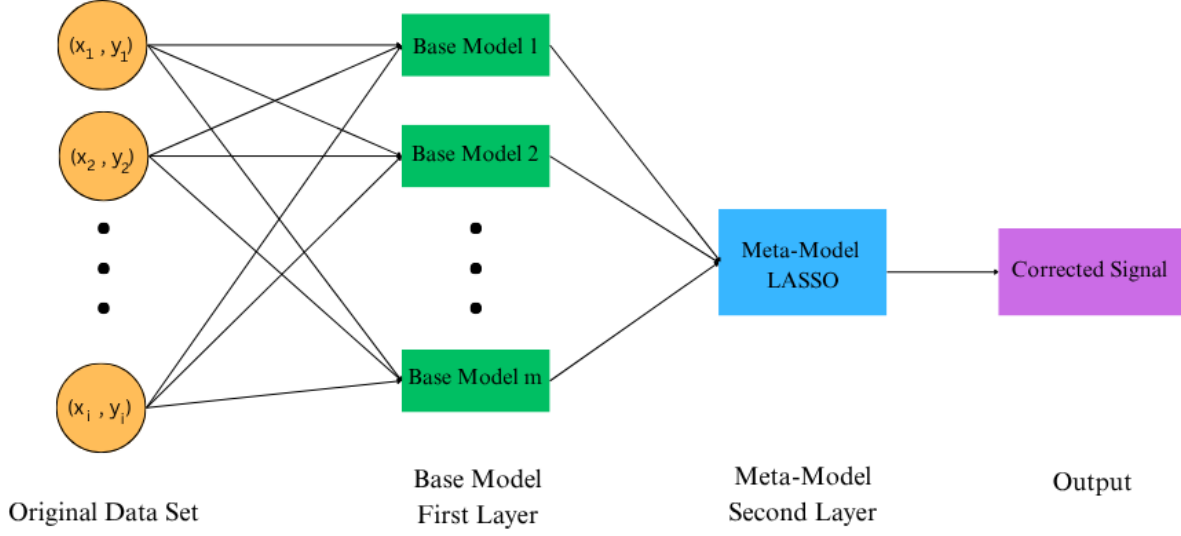


Figure 4: Two-layer stacking to account for the effects of temperature. The first layer applies three SVMs base models with different parameters and kernels and an XGBoost which are combined in a second layer with a LASSO algorithm.

The first layer is designed to fit the original input data originating from different data structures (e.g., displacement, inclination) and spatial perspectives (e.g. different physical sensor locations). This layer uses multiple base models using two learning algorithms: **SVM** (Support Vector Machine) and **XGBoost** (Extreme Gradient Boosting):

1. **XGBoost** is applied in one base model to capture complex nonlinear relationships in the data.
2. **SVM** is applied in three base models, each with a different kernel function (**linear**, **rbf**, and **poly**) to approximate the target values within a specified error margin.

Hyperparameter tuning via grid search with cross-validation is used to select the optimal parameters for each **SVMs**. Each of these four algorithms is trained on the data of each sensor individually. Thus, the number of base models is equal to $4 \times C$. To mitigate overfitting and information leakage, K -fold cross-validation with $K = 5$ is also applied when training each basic model.

The meta-model framework consists of the following steps:

- The original feature dataset $\mathbf{S}$ is divided into a training set (80%) and a test set (20%). The set of input features x_i in the training set is denoted $\mathbf{R}^*$, while $\mathbf{R}$ consists of the set of input features in the test set. The set of target values y_i in the training and test sets are denoted as $\mathbf{y}^*$ and $\mathbf{y}$ respectively. For this division the function `TimeSeriesSplit` in `scikit-learn` is used to provide train/test indices to split time-ordered data, since other cross-validation methods lead to training on future data and evaluating on past data. The set $\mathbf{R}^*$ is divided into subsets $\mathbf{B}_k$ during cross-validation: $\mathbf{R}^* = [\mathbf{B}_1, \dots, \mathbf{B}_k]$.
- The sets $\mathbf{B}_k^* = \mathbf{R}^* \setminus \mathbf{B}_k$ are used as training sets of the k -th fold cross validation while the test sets are $\mathbf{B}_k$. After all training is performed, the prediction value of the j -th basic model, where $j \in \{1, \dots, 4 \times C\}$, on the test set $\mathbf{B}_k$ is obtained and denoted as $F_j(\mathbf{B}_k)$. On the other hand, the prediction value of such model on the separate hold-out test set $\mathbf{R}$ is denoted as $F_{j,k}(\mathbf{R})$. Each $F_{j,k}$ is trained on a different subset of $\mathbf{R}^*$, none of them has seen any part of $\mathbf{R}$ during training, so applying all fold-specific models to $\mathbf{R}$ does not result in data leakage.
- Once all predictions ($K \times 4 \times C$ in total) have been performed the new feature dataset $\mathbf{S}^{\text{new}}$ that serves as input of the Meta-Model is constructed as $\mathbf{S}^{\text{new}} = [\text{Train}, \text{Test}] = [(\mathbf{M}^*, \mathbf{y}^*), (\mathbf{M}, \mathbf{y})]$. Where $\mathbf{y}^*$ is the label data of the original training set and $\mathbf{y}$ is the label data of the original test set.

- The training set of the meta model $\mathbf{M}^*$ is assembled as $\mathbf{M}^* = [\mathbf{M}_1^*, \dots, \mathbf{M}_m^*]$ where

$$\mathbf{M}_j^* = [F_j(\mathbf{B}_1), F_j(\mathbf{B}_2), \dots, F_j(\mathbf{B}_K)],$$

for $j = 1, \dots, m$. This means only out-of-fold predictions $\mathbf{M}^*$ obtained from K-fold cross-validation on the training set $\mathbf{R}^*$ will be used for training the meta model.

- The test set of the meta model $\mathbf{M}$ is calculated as $\mathbf{M} = [\mathbf{M}_1, \dots, \mathbf{M}_j]$ with:

$$\mathbf{M}_j = \frac{1}{K} \sum_{k=1}^K \gamma_{j,k} \cdot F_{j,k}(\mathbf{R}), \quad (18)$$

where $\gamma_{j,k}$ are weights that combine both accuracy and temporal weights that are described in the Appendix 7.1. Hence, after training in the set $\mathbf{R}^*$, the base models $F_{j,k}$ are applied to the entire test set $\mathbf{R}$, and a weighted average is computed to be used as the test set of the meta-model.

Second layer (Meta-Model):

The LASSO (least absolute shrinkage and selection operator) algorithm is used as the framework for the second layer of the ensemble. This algorithm combines the results from the different base-models from the first layer and helps to avoid collinearity and overfitting. The steps are described below:

- The output of the ensemble's first layer (as described in 3.2) is of the form $\mathbf{S}^{\text{new}} = [(\mathbf{M}^*, \mathbf{y}^*), (\mathbf{M}, \mathbf{y})]$.
- For each sensor $c = 1, \dots, C$, a separate LASSO regression model is trained using only the predictions from that sensor's base models. Let T_{train} denote the time indices of the training set and let $\mathbf{y}_c^* \in \mathbb{R}^{T_{\text{train}}}$ be the scaled target signal for sensor c . Then $\mathbf{M}_c^* \in \mathbb{R}^{T_{\text{train}} \times 5}$ is defined as the meta-model training set matrix for sensor c , constructed by stacking the out-of-fold predictions from its four base models (three SVRs with different kernels and one XGBoost as described in 3.2). The LASSO model then finds the coefficients β in the linear fitting $\mathbf{y}_c^* = \mathbf{M}_c^* \cdot \beta + \varepsilon$, with ε the associated error, following the optimization problem 19:

$$\hat{\beta}^{(c)} = \arg \min_{\beta} \left(\|\mathbf{y}_c^* - \mathbf{M}_c^* \cdot \beta\|_2^2 + \alpha^{\text{lasso}} \|\beta\|_1 \right), \quad (19)$$

where $\|\cdot\|_2$ and $\|\cdot\|_1$ denote the L^2 and L^1 -norms respectively.

- The regularization parameter α^{lasso} is selected via nested cross-validation on $\mathbf{R}^*$. The LASSO coefficients $\hat{\beta}^{(c)} \in \mathbb{R}^5$ are used for verification on the test set of the meta-model $(\mathbf{M}, \mathbf{y})$ by calculating the **residual** between the prediction $\mathbf{M}_c \hat{\beta}^{(c)}$ with $\mathbf{y}_c$.
- The trained LASSO is then applied to the full feature dataset using predictions from the fully-trained base models. The corrected signal $\hat{\mathbf{Y}} = [\hat{\mathbf{Y}}_1, \dots, \hat{\mathbf{Y}}_C]$ across the entire time range is obtained as a linear combination of these predictions using the sensor specific LASSO coefficients $\hat{\beta}^{(c)}$ learned during training 20:

$$\hat{\mathbf{Y}}_c = \begin{bmatrix} M_c^* \\ M_c \end{bmatrix} \hat{\beta}_c \quad (20)$$

The final output is a corrected signal that removes the temperature-induced components from the original sensor measurements.

3.3 Illustrative Results

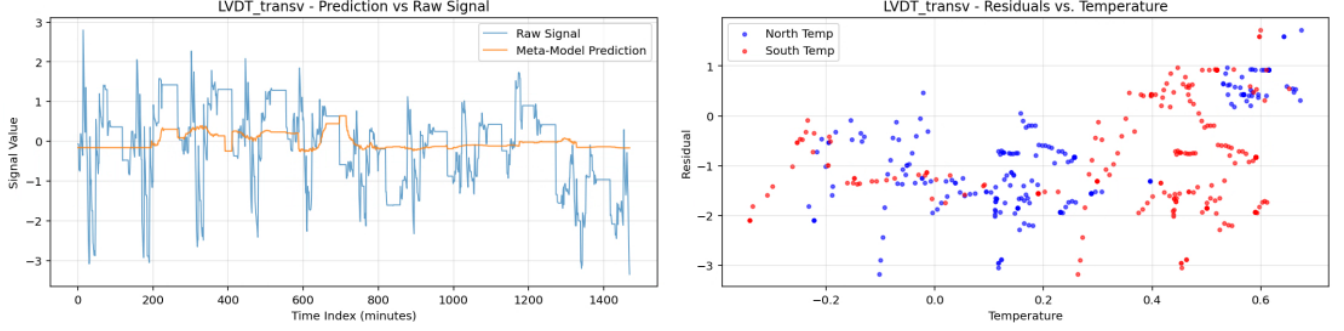


Figure 5: Correction to the scaled raw signal from the transversal LVDT sensor for the pot bearing P5 applying the two-layer stacking algorithm discussed in 3.2 to mitigate fluctuations caused by temperature. The scaled residuals $\mathbf{y} - \mathbf{M}\hat{\beta}^{\text{lasso}}$ are also included. The regularization parameter found via nested cross validation is $\alpha^{\text{lasso}} = 0.0359$

These results indicate that signal fluctuations were successfully reduced; however, the meta-model proved to be overly complex and failed to effectively capture the influence of temperature. This is evidenced by persistent linear trends in the residuals and the output signal becoming nearly flat, suggesting the model may have overfit or failed to generalize. This limitation may arise from various factors, with the primary one likely being the 10-minute sampling rate, which significantly reduced the amount of usable data. Alternative approaches such as **Principal Component Analysis** (PCA) [6], or **Auto-Regressive Integrated Moving-Average** (ARIMA) [7] models were discussed and may have been more suitable for this task, given the nature of the data and the modeling objectives. This aspect should be revisited in future work, as properly addressing data drift is essential for the subsequent stages of anomaly detection.

4 Structural Anomaly Detection via the TimesBlock Architecture

The output from the data cleansing process described in Section 3 provides a multivariate time series $\mathbf{X}_{1D} \in \mathbb{R}^{T \times C}$, where T denotes the number of time steps and C is the number of sensor channels (e.g., displacement and inclination). Fluctuations induced by environmental temperature effects have been mitigated using the two-layer stacking ensemble approach detailed in Section 3.2, or other previously mentioned approaches such as PCA or ARIMA. This section, which closely follows the methodology outlined in [8] and [9], presents a method to map the cleaned 1D time series into a 2D tensor representation, which enables more effective extraction of temporal patterns for unsupervised anomaly detection.

Motivation:

Traditional 1D representations of time series data often struggle to simultaneously capture both short-term and long-term temporal dependencies. To address this limitation, the 1D input $\mathbf{X}_{1D}$ is reshaped into a set of 2D tensors that preserve both intra-period and inter-period variations. This 2D representation facilitates the use of convolutional architectures and improves the model’s ability to detect structural anomalies by explicitly encoding periodic patterns in the spatial layout of the data.

4.1 Mathematical Background

Given a multivariate discrete time series $\{\mathbf{X}_t\}_{t=0}^{T-1}$, where each $\mathbf{X}_t \in \mathbb{R}^C$ represents measurements from C channels at time step t , the Discrete Fourier Transform (DFT) of each channel $c = 1, \dots, C$ is defined as [10]:

$$\hat{\mathbf{X}}_k^{(c)} = \sum_{t=0}^{T-1} \mathbf{X}_t^{(c)} \cdot e^{-i2\pi kt/T}, \quad k = 0, 1, \dots, T-1 \quad (21)$$

where $\hat{\mathbf{X}}_k^{(c)}$ is the complex-valued Fourier coefficient for frequency index k in channel c . The magnitude $|\hat{\mathbf{X}}_k^{(c)}|$ indicates the amplitude of the corresponding frequency component, while the phase $\arg(\hat{\mathbf{X}}_k^{(c)})$ encodes the phase shift. This transformation maps the signal from the time domain to the frequency domain, revealing periodic structures that are not easily tractable when considering the time series alone as described in Figure 6.

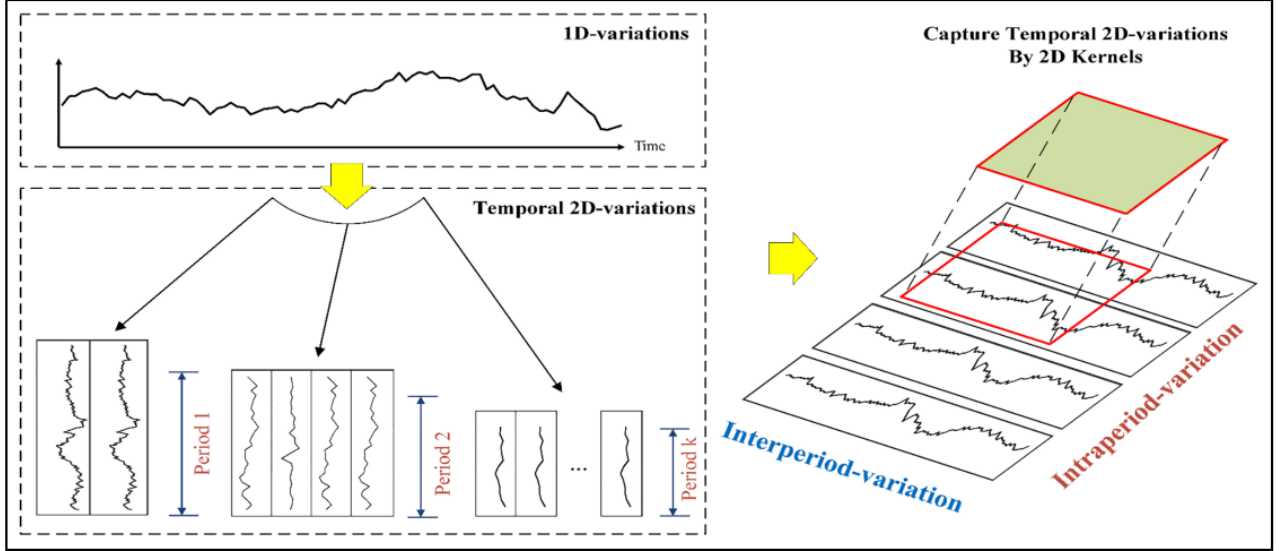


Figure 6: Illustration of how the Discrete Fourier Transform maps 1D time series data to 2D data, allowing for modeling of both intra-period and inter-period variations. Figure taken from [5].

The Fast Fourier Transform (FFT) is an efficient algorithm that determines the DFT by exploiting symmetries in the computation. The function `fft` from the `scipy.fft` module is used to perform the FFT and compute the corresponding amplitudes. Then, the average is computed along each column (each channel/sensor entry), returning a vector of the shape $\mathcal{A} \in \mathbb{R}^T$:

$$\mathcal{A} = \text{Average}(\text{Amplitude}(\text{FFT}(\mathbf{X}_{1D}))) = \left\{ \frac{1}{C} \sum_{c=1}^C |\hat{\mathbf{X}}_k^{(c)}| : k = 0, \dots, T-1 \right\} \quad (22)$$

Next, the k dominant frequency components are selected based on the amplitudes $\mathcal{A}$ computed in the previous step (Equation 22). Specifically, the goal is to identify the k integer-valued positive frequencies that correspond to the highest average amplitudes, excluding the zero-frequency term (DC component). The function `arg_topk_A` in the Python script is used to extract the indices of these top- k frequency components, along with their associated amplitudes $\{A_1, \dots, A_k\}$. Formally, the selected frequencies are given by:

$$\{f_1, \dots, f_k\} = \underset{f_i \in \{1, \dots, \lfloor \frac{T}{2} \rfloor\}}{\text{arg Top } k} (\mathcal{A}). \quad (23)$$

The corresponding period lengths p_i from the first k dominant frequencies are also selected:

$$p_1, \dots, p_k = \left\lceil \frac{T}{f_1} \right\rceil, \dots, \left\lceil \frac{T}{f_k} \right\rceil. \quad (24)$$

4.2 Inception Block

In deep convolutional neural networks (CNNs), feature extraction at multiple scales is crucial to effectively capture local and global patterns in the data. A widely adopted approach is the use of parallel convolutions, which form the core idea behind **inception functions** [11].

Parallel convolutions refer to the application of multiple convolutional operations in parallel to the same input tensor, where each operation uses a different kernel size or transformation. In the Python script the function `BasicInceptionBlock2D` performs four parallel branches with different convolution types: (1×1, 3×3, 5×5, and pooling) by applying the function PyTorch function `Conv2d`. Each branch extracts features at different receptive field sizes, which enables the network to learn both fine and coarse patterns simultaneously. **ReLU** is applied to the output of each branch to introduce nonlinearities which allow the network to learn complex behaviors. After the parallel convolutions, the output tensors from all branches are concatenated along the channel dimension using the function `torch.cat` from PyTorch.

```
branch1 = Conv2d(in, out//4, kernel=1) -> ReLU(branch1)
branch3 = Conv2d(in, out//4, kernel=1) -> Conv2d(out//4, out//4, kernel=3) -> ReLU(branch3)
branch5 = Conv2d(in, out//4, kernel=1) -> Conv2d(out//4, out//4, kernel=5) -> ReLU(branch5)
branch_pool = MaxPool2d(...) -> Conv2d(in, out//4, kernel=1) -> ReLU(branch_pool)
output = concat([ReLU(branch1), ReLU(branch3), ReLU(branch5), ReLU(branch_pool)])
```

4.3 TimesBlock Module

The TimesBlock module is designed to extract K two-dimensional representations from the original multi-variate time series to enhance its modeling capacity. It operates in multiple layers to progressively refine the temporal representation and facilitate anomaly detection. Deep learning methods such as TimesNet enable the extraction of multiscale temporal patterns [8], which are particularly valuable for detecting small and localized anomalies in sensor networks, such as those used in structural health monitoring (SHM). Through a series of layers, it progressively refines the temporal features and enhances anomaly detection capabilities.

As a first step the one-dimensional input time series $\mathbf{X}_{1D} \in \mathbb{R}^{T \times C}$ is projected to a higher dimensional space of size d_{model} with $\mathbf{X}_{1D}^0 \in \mathbb{R}^{T \times d_{\text{model}}}$ and $d_{\text{model}} \geq C$ using the function `embedding` in the Python script. This serves as the input for the first layer. Let k be the number of dominant frequencies to be extracted at

each layer. The TimesBlock module will be applied several times in layers:

For the l -th layer of the model architecture, the input is denoted as $\mathbf{X}_{1D}^{l-1} \in \mathbb{R}^{T \times d_{\text{model}}}$ and the output $\mathbf{X}_{1D}^l \in \mathbb{R}^{T \times d_{\text{model}}}$ is given by:

$$\mathbf{X}_{1D}^l = \text{TimesBlock}(\mathbf{X}_{1D}^{l-1}) + \mathbf{X}_{1D}^{l-1}. \quad (25)$$

TimesBlock Computation:

As detailed in [9], each TimesBlock applies the following sequence of operations, summarized in Figure 7:

- The one-dimensional embedded time series is transformed into a two-dimensional space to obtain a set of two-dimensional tensors:

$$\mathbf{X}_{2D}^{l,i} = \text{Reshape}_{p_i, f_i}(\text{Padding}(\mathbf{X}_{1D}^{l-1})), \quad (26)$$

for $i = 1, \dots, k$ and $\mathbf{X}_{2D}^{l,i} \in \mathbb{R}^{p_i \times f_i \times d_{\text{model}}}$.

- An **inception function** as described in Section 4.2 is applied to the input:

$$\hat{\mathbf{X}}_{2D}^{l,i} = \text{Inception}(\mathbf{X}_{2D}^{l,i}), \quad (27)$$

for $i = 1, \dots, k$.

- The features $\hat{\mathbf{X}}_{2D}^{l,i}$ are transformed into a one-dimensional space $\hat{\mathbf{X}}_{1D}^{l,i} \in \mathbb{R}^{T \times d_{\text{model}}}$ to facilitate the aggregation of the collected information (**reshape1D**):

$$\hat{\mathbf{X}}_{1D}^{l,i} = \text{Trunc}(\text{Reshape}_{1, (p_i \times f_i)}(\hat{\mathbf{X}}_{2D}^{l,i})), \quad (28)$$

for $i = 1, \dots, k$.

- Based on the average amplitudes $\{A_1, \dots, A_k\}$ obtained in the first line, the **Softmax** function is applied to obtain the corresponding weights for the k terms:

$$w_i = \frac{e^{A_i}}{\sum_{i=1}^k e^{A_i}}. \quad (29)$$

The output of the TimesBlock module becomes:

$$\mathbf{X}_{1D}^l = \sum_{i=1}^k w_i \cdot \hat{\mathbf{X}}_{1D}^{l,i} + \mathbf{X}_{1D}^{l-1}. \quad (30)$$

- For the last layer in the architecture, the function **nn.Linear** from PyTorch returns the final output from $\mathbb{R}^{T \times d_{\text{model}}}$ to size $\mathbb{R}^{T \times C}$ by applying a linear transformation.

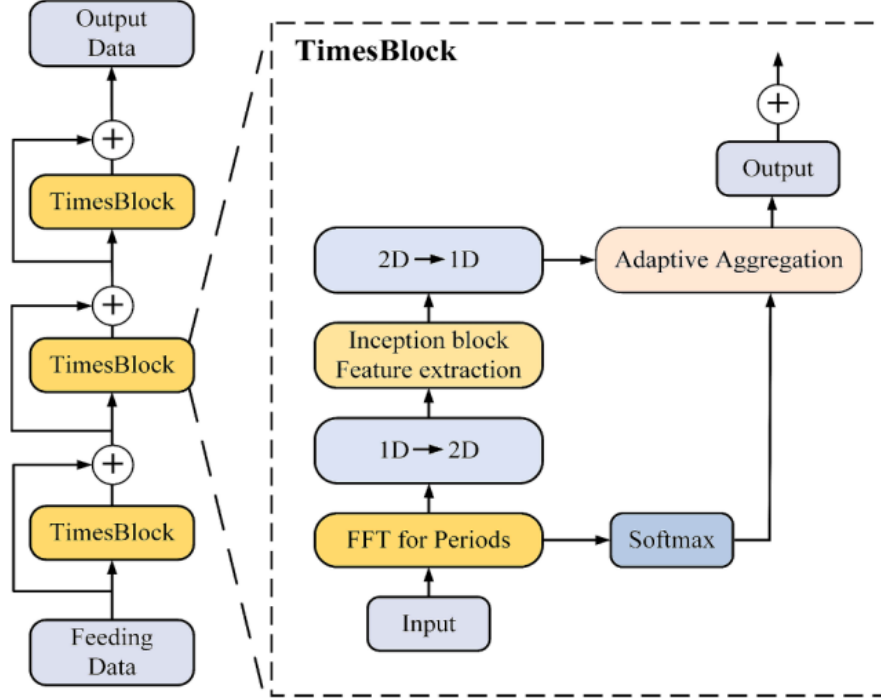


Figure 7: Schematic illustration of the TimesBlock module and its layered application for periodic pattern extraction. Figure retrieved from [9].

Training Details:

The model is trained over 10 **epochs**, where each epoch represents one full training iteration over the input data. Training is performed by minimizing the MSE loss (Equation 31) using backpropagation and a gradient-based optimizer, in this case Adam optimizer, a widely used default optimizer in deep learning applications [12]. A batch size of 1 is used due to the limited dataset size, but this can be adjusted based on computational resources or model requirements.

The training procedure involves the following steps:

1. The input embedded sequence $\mathbf{X}_{1D}^0 \in \mathbb{R}^{T \times d_{\text{model}}}$ is passed on through the TimesBlock network over L layers, producing a reconstructed output.
2. The reconstruction loss $\mathcal{L}(\theta)$ is computed using the Mean Squared Error (MSE):

$$\mathcal{L}(\theta) = \frac{1}{T \cdot C} \sum_{t=1}^T \sum_{c=1}^C [(\mathbf{X}_{1D}^L)_{t,c} - (\mathbf{X}_{1D})_{t,c}]^2. \quad (31)$$

3. Gradients of the loss (Equation 31) with respect to the model parameters (weights and biases in the convolutional and linear layers) are computed via backpropagation. The parameters are updated using the Adam optimizer:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta), \quad (32)$$

where η is the learning rate, which can be tuned for better convergence.

The final output after multiple TimesBlock transformations is a refined time series reconstruction that better captures the underlying periodic patterns in the SHM data. This strengthens anomaly detection, where deviations in reconstruction (high MSE) are indicative of abnormal behavior.

4.4 Anomaly Detection via Extreme Value Analysis

Extreme Value Analysis (EVA) is a statistical technique used to model the behavior of extreme deviations within a dataset. Contrary to traditional Gaussian-based methods, EVA focuses on tail behavior, making it especially suited for anomaly detection in structural health monitoring (SHM) applications, where critical anomalies often lie in the extremes of the error distribution [9].

In this work, the reconstruction error of the TimesBlock architecture (Section 4.3) is used as the metric for anomaly detection [9]. Data points exhibiting high MSE values are flagged as potential anomalies since they indicate significant deviations from the expected patterns encoded in the model during training.

The tail of the reconstruction error distribution, obtained from the mean squared error between the original signal $\mathbf{X}_{1D}$ (assumed to represent normal operating conditions and used for training the TimesBlock architecture) and the final output of the TimesBlock architecture $\mathbf{X}_{1D}^L$, is modeled using a **Generalized Pareto Distribution** (GPD). This has an implicit theoretical simplification, since the observations are assumed independent and identically distributed, while SHM data is often correlated over time. The estimated quantiles of the GPD are then used to define anomaly detection thresholds corresponding to a chosen significance level, therefore separating outliers from normal variations without requiring labeled data.

Selecting an appropriate threshold is critical in the step of fitting a Generalized Pareto Distribution (GPD) to the tail of the reconstruction error distribution. If the threshold is too high, there may be too few exceedances to estimate the distribution reliably. If it is too low, the GPD assumption may no longer hold, leading to underestimated tail risks and reduced anomaly detection accuracy. A careful balance is therefore required to ensure both statistical validity and model stability.

For a random variable following a Generalized Pareto Distribution $\mathbf{X} \sim \text{GPD}(\mu, \sigma, \zeta)$ the probability density function is given by:

$$G(x; \mu, \sigma, \zeta) = \begin{cases} 1 - \left(1 + \zeta \frac{x - \mu}{\sigma}\right)^{-1/\zeta}, & \zeta \neq 0 \\ 1 - \exp\left(-\frac{x - \mu}{\sigma}\right), & \zeta = 0 \end{cases} \quad (33)$$

with μ the position parameter, σ is the scale parameter, and ζ the shape parameter. The probability density function of the GPD is shown in Figure 8.

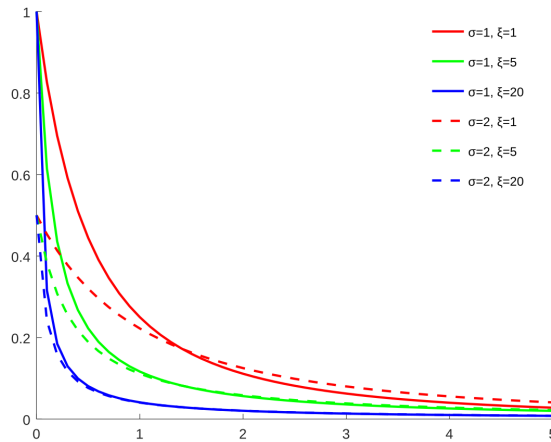


Figure 8: Probability density functions of the Generalized Pareto Distribution for $\mu = 0$ with varying scale σ and shape ζ parameters. Image retrieved from [13].

The method implemented for anomaly detection closely follows the general methodology described in [14].

- The mean squared error between the corrected signal $\mathbf{X}_{1D}$ and the final output of the TimesBlock algorithm [L -th layer] $\mathbf{X}_{1D}^L \in \mathbb{R}^{T \times C}$ along each temporal entry is computed. Since the mean squared error is over all the channels C , this method is detecting **global anomalies** (i.e. deviations that occur collectively in all the sensors). Error detection in a single sensor can be easily implemented by selecting the corresponding column from $\mathbf{X}_{1D}$ and $\mathbf{X}_{1D}^L \in \mathbb{R}^{T \times C}$. The following methods can then be applied with $C = 1$. The errors are aggregated into a vector $\mathcal{E} = \{e_1, \dots, e_T\} \in \mathbb{R}^T$ where:

$$e_t = \frac{1}{C} \sum_{c=1}^C [(\mathbf{X}_{1D}^L)_{t,c} - (\mathbf{X}_{1D})_{t,c}]^2 \quad (34)$$

here t denotes each temporal entry and c denotes each individual channel/sensor.

- The $p\%$ percentile of the error data, denoted e_p , is computed using the function `scoreatpercentile` from the `scipy.stats` module. By definition this is the value e_p such that $P(e_i \leq e_p) = p$ for any entry $e_i \in \{e_1, \dots, e_T\} = \mathcal{E}$.
- The error entries $e_i \in \mathcal{E}$ such that $e_i \geq e_p$ are selected. For these values the excess v_i is determined following $v_i = e_i - e_p$ and are stacked in the excess vector $\mathcal{V}$.
- The excess vector $\mathcal{V}$ is expected to follow a Generalized Pareto Distribution (Equation 33). The distribution is fitted to the excesses by applying the Python function `genpareto.fit` to estimate the parameters $\hat{\mu}, \hat{\sigma}, \hat{\zeta}$.
- A significance level α is then established. The value y_α such that $P(\mathcal{V} \leq y_\alpha) = \alpha$ with $\mathcal{V} \sim \text{GPD}(\hat{\mu}, \hat{\sigma}, \hat{\zeta})$ is determined by applying the percent point function (inverse cumulative distribution function) using the built-in function `genpareto.ppf` from `scipy.stats`.
- The anomaly detection threshold τ_α computed from the GPD with significance α is given by $\tau_\alpha = e_p + y_\alpha$. Once the threshold has been established, it remains fixed.
- For any new and potentially anomalous input signal, the **previously trained** TimesBlock architecture is applied to reconstruct the signal, and the corresponding reconstruction errors $\hat{e}_t$ are computed at each time step using Equation 34. If the error $\hat{e}_t$ at any time step t exceeds the previously established anomaly threshold τ_α then that time step $t \in \{1, \dots, T\}$ is flagged as anomalous.

4.5 Illustrative Results and Practical Limitations

Simulating a real deployment scenario, the temperature-corrected signal from an inclinometer (i.e., a single sensor: $C = 1$) was divided into training and test sets. The mean and standard deviation computed from the training data were used to normalize both sets, thereby preventing data leakage. The TimesBlock architecture was trained on the training set using three layers, an embedding dimension of $d_{\text{model}} = 256$, and a total of ten epochs. The mean squared reconstruction error between the original signal and the output of each TimesBlock layer at the final epoch is shown in Figure 9, demonstrating a progressive reduction in error across the layers. The mean squared error from the last layer was used to define the threshold for extreme values following the procedure described in Section 4.4. Additionally, the training loss $\mathcal{L}(\theta)$ over the ten epochs is plotted in Figure 10. The plot shows a consistent downward trend in the training loss, indicating stable convergence of the model.

Finally, Figure 11 presents the normalized signal used for testing, along with the corresponding mean squared error (MSE) obtained by applying the pre-trained TimesBlock architecture to the test data. The previously established anomaly detection threshold is also shown, enabling a visual assessment of the model’s ability to distinguish anomalous behavior. The results indicate noticeable spikes in the mean squared error (MSE) around regions of abrupt changes in the original signal. This is an expected outcome since such fluctuations may reflect potential anomalies. However, these spikes remain below the established threshold, suggesting that the observed variations do not significantly deviate from the learned normal behavior in the training data to be classified as abnormal.

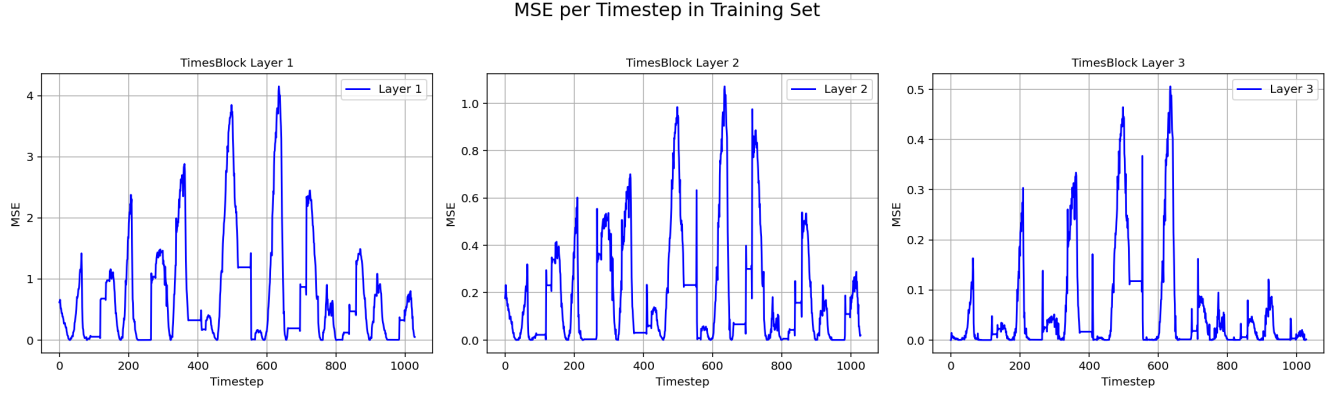


Figure 9: Mean squared error between a normalized inclinometer signal (y -axis inclination from the Pot Bearing P1) and its reconstruction over each layer of the TimesBlock architecture. The parameters involved are $k = 3$, $d_{\text{model}} = 256$ and $\eta = 0.001$.

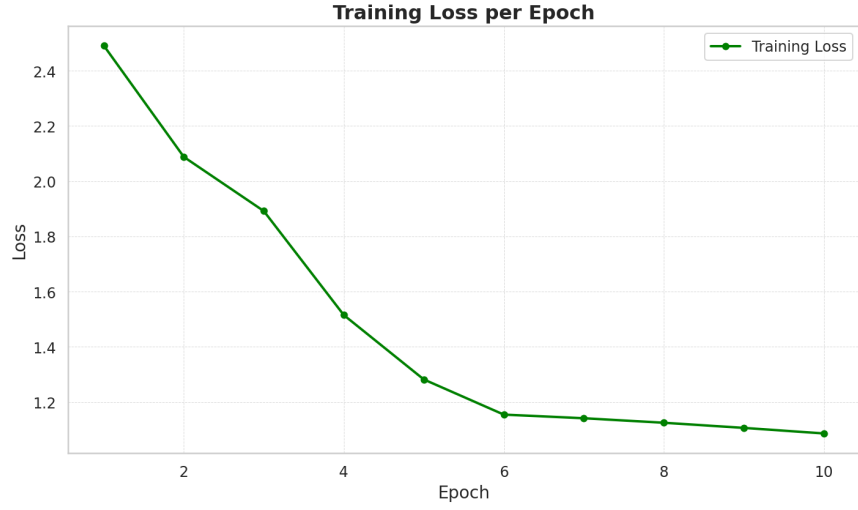


Figure 10: Training loss $\mathcal{L}(\theta)$ over the TimesBlock defined above, with a learning rate $\eta = 0.001$. The trend shows a clear descend over epochs, indicating stable convergence of the model during the training phase.

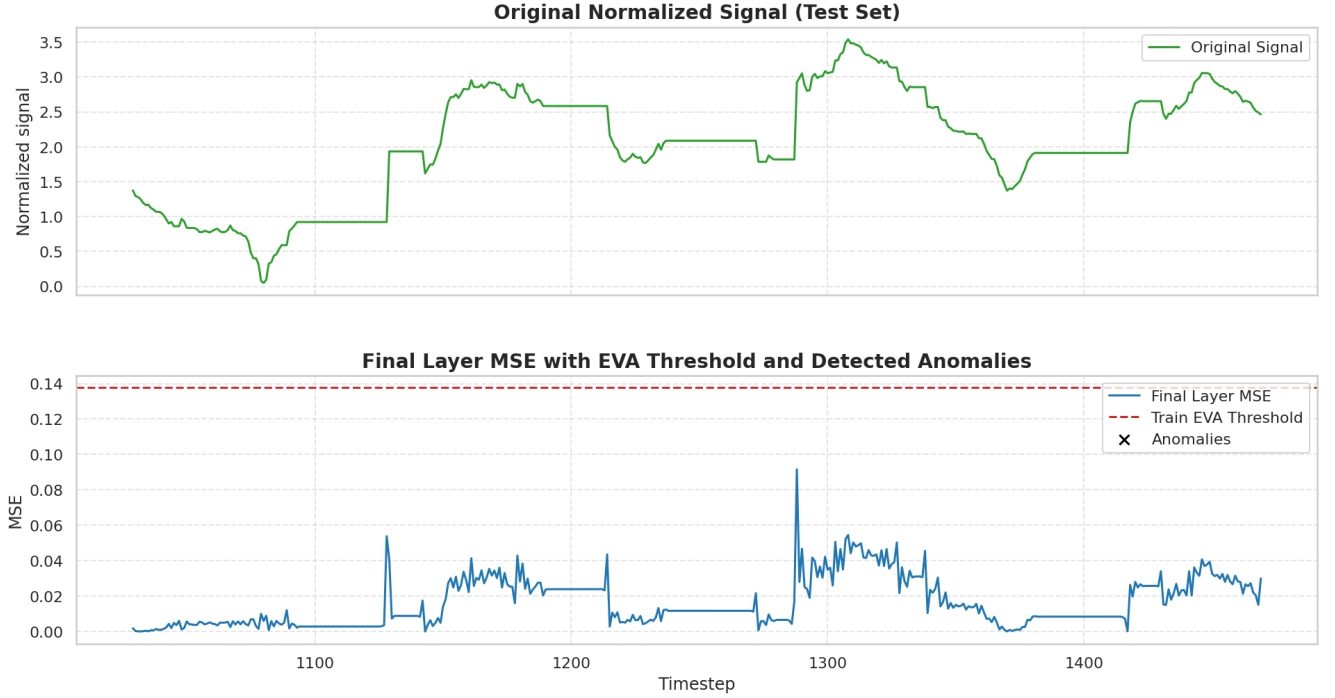


Figure 11: Normalized test signal alongside the mean squared error (MSE) computed using the pre-trained TimesBlock model. The anomaly detection threshold is indicated for reference. Although spikes in MSE occur near abrupt signal changes, they remain below the threshold, indicating no significant anomaly following the criterion detailed in Section 4.4.

Practical Limitations

The proposed TimesBlock and Extreme Value Analysis (EVA) framework shows promising results, but also faces several limitations:

- **Dependence on parameter tuning:** The selection of parameters such as the number of layers, embedding dimension d_{model} , and number of dominant frequencies k can significantly affect performance. Poor tuning may lead to underfitting or overfitting.
- **Assumption of stationarity:** The EVA framework assumes that the distribution of extreme errors is stable over time. In practice, evolving operational or environmental conditions might violate this assumption. As such, the GPD may need to be adapted on different time windows.
- **Computational cost:** The multi-layer TimesBlock architecture with several frequency extraction steps and convolutional blocks can be computationally intensive for real-time monitoring in large-scale systems.
- **Variability in GPD Fitting:** Minor fluctuations in the error distribution from the TimesBlock output signal can produce variations in the estimated parameters of the GPD. This manifests as inconsistent quantile estimates for the anomaly detection threshold across multiple runs, even when the model architecture and parameters remain fixed. To improve robustness, one possibility is to perform repeated GPD fittings and aggregate the resulting thresholds by averaging.
- **Training data may contain anomalies:** A fundamental assumption of the proposed approach is that the training dataset is free of anomalies and represents only the normal operating behavior of the structure, which may not hold in SHM scenarios. If anomalies are present in the training data, the model may learn to treat them as normal behavior, leading to reduced sensitivity during inference and increased false negatives.

5 Model Validation Framework

To evaluate the model’s ability to detect anomalies, synthetic anomalies are injected into the data by adding white noise, following the procedure described in [9]. Once the modified dataset is generated, ground-truth labels are assigned to the affected samples in the test set. Model performance is then assessed by comparing the predicted anomalies against the ground-truth labels in the test set.

5.1 Evaluation Procedure

A clean time-series sensor dataset $\mathbf{X} \in \mathbb{R}^{T \times C}$, for which environmental effects have been mitigated (e.g., using the two-layer stacking ensemble described in Section 3.2), is split into training and test sets. The TimesBlock architecture is trained on the training set, and the reconstruction error is computed following Equation 34. A threshold is then established following the procedure outlined in Section 4.4.

Next, the test signal is augmented using **stochastic numerical simulations** via the noise injection method described in Section 5.2. The pre-trained TimesBlock model is then applied to the augmented test data, and the mean squared error (MSE) is calculated at each time step. If the MSE at a given time step exceeds the previously established threshold, the point is flagged as an anomaly.

5.2 Noise addition for anomaly simulation

This section describes the introduction of Gaussian white noise, modeling sensor failure for validation, following a similar approach to that introduced in [9]. Noise is added to a clean multivariate time-series $\{\mathbf{X}_t\}$ as follows:

- An **ensemble instance** refers to one realization of added noise applied to the original signal. The total number of ensemble instances N is fixed. For each sensor $j \in \{1, \dots, C\}$ and each ensemble instance $i \in \{1, \dots, N\}$, a noise standard deviation value $(\sigma_{i,j})$ is generated:

$$\sigma_{i,j} = \sigma_j^{\min} + (\sigma_j^{\max} - \sigma_j^{\min}) \cdot u_{i,j} \quad u_{i,j} \sim \mathcal{U}(0, 1) \quad (35)$$

where $\sigma_j^{\min}$ and $\sigma_j^{\max}$ are fixed parameters and $u_{i,j}$ is a random value sampled from the uniform distribution $\mathcal{U}(0, 1)$. The output is a matrix $\Sigma = [\sigma_{i,j}] \in \mathbb{R}^{N \times C}$.

- A standard Gaussian distribution is then used to sample a matrix of raw noise:

$$\mathbf{Z}_i \sim \mathcal{N}_{(T \times C)}(0, 1) \quad (36)$$

for each ensemble instance $i \in \{1, \dots, N\}$. To normalize the raw noise, the sample mean $\mu_{i,j}$ and the sample standard deviation $s_{i,j}$ along each column (each sensor j) of $\mathbf{Z}_i = [\mathbf{Z}_{i,1}, \dots, \mathbf{Z}_{i,C}]$ is calculated and the noise standardized accordingly:

$$\mathbf{W}_i[:, j] = \left(\frac{\mathbf{Z}_i[:, j] - \mu_{i,j}}{s_{i,j}} \right) \cdot \sigma_{i,j}, \quad \forall j \in \{1, \dots, C\} \quad (37)$$

The final output is a list of N matrices:

$$\mathcal{W} = \{\mathbf{W}_1, \mathbf{W}_2, \dots, \mathbf{W}_N\}, \quad \mathbf{W}_i \in \mathbb{R}^{T \times C} \quad (38)$$

which represent sensor-specific white noise samples scaled according to the fixed standard deviation bounds.

- The noise matrices are averaged element-wise to obtain a mean perturbation matrix:

$$\bar{\mathbf{W}} = \frac{1}{N} \sum_{i=1}^N \mathbf{W}_i$$

- For each time index $t \in \{1, \dots, T\}$, the original signal is perturbed by adding a scaled noise vector as follows:

$$\mathbf{X}[t, :] \rightarrow \mathbf{X}[t, :] + \lambda \cdot \bar{\mathbf{W}}[t_0, :],$$

where λ is a fixed scaling factor and $\bar{\mathbf{W}} \in \mathbb{R}^{T \times C}$ is a noise matrix. If the perturbation $\lambda \cdot \bar{\mathbf{W}}[t_0, :]$ exceeds a predefined threshold in magnitude, the corresponding data point is labeled as abnormal (label 1); otherwise, it is labeled as normal (label 0). This labeling yields a binary vector $\mathbf{b} \in \{0, 1\}^T$ representing the ground-truth labels.

5.3 ROC curve and F1-Score

The Receiver Operating Characteristic (ROC) curve is a graphical tool used to assess the performance of a binary classifier. It plots the true positive rate against the false positive rate at various decision thresholds, illustrating the trade-off between detecting true positives and avoiding false alarms [15]. The area under the ROC curve (AUC) provides a single scalar value summarizing the classifier’s performance across all thresholds: a value of 1 indicates perfect discrimination, while 0.5 corresponds to random guessing. The F_1 -score, derived from the model’s precision and recall, is used to evaluate classification performance under **class imbalance**.

Precision is defined as the proportion of correctly predicted positive instances (true positives TP) among all instances predicted as positive, including false positives (FP). Recall is the proportion of true positives among all actual positive instances, including those the model failed to identify (false negatives FN) [15]:

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}. \quad (39)$$

The F_1 -score is then calculated as:

$$F_1 = \frac{2PR}{P + R} \quad (40)$$

This metric is particularly useful when the costs of false positives and false negatives are similar, and when a balance between precision and recall is desired.

5.4 Illustrative Results and Practical Limitations

A temperature-corrected signal from an LVDT sensor (i.e. only one channel, $C = 1$) was split into training (70%) and test (30%) sets. The training data was normalized, and the same normalization parameters were applied to the test set to prevent data leakage. The TimesBlock architecture was then trained on the training data, assuming it to represent normal (anomaly-free) bridge behavior. The model was configured with $k = 3$ (the number of dominant frequencies to be extracted at each layer of the TimesBlock), $d_{\text{model}} = 128$, three stacked TimesBlock layers, a learning rate of $\eta = 0.01$, and trained for 10 epochs. After training, the reconstruction error between the model output and the original training signal was used to estimate an anomaly detection threshold τ_α via extreme value analysis as described in Section 4.4. Then, noise was injected to the test data using the procedure described in Section 5.2, which yields a binary vector $b \in \{0, 1\}^T$ containing the ground-truth indices where there is a synthetic anomaly in the data set. The previously trained TimesBlock model is applied to the noisy signal, and the reconstruction error e_t computed at each time step. If the reconstruction error exceeds the threshold τ_α , that time instance is classified as an anomaly. The resulting predictions formed another binary vector, which was then compared against the ground-truth labels to assess model performance.

Figure 12 presents the LVDT sensor signal after the injection of noise and synthetic anomalies using the procedure described in Section 5.2.

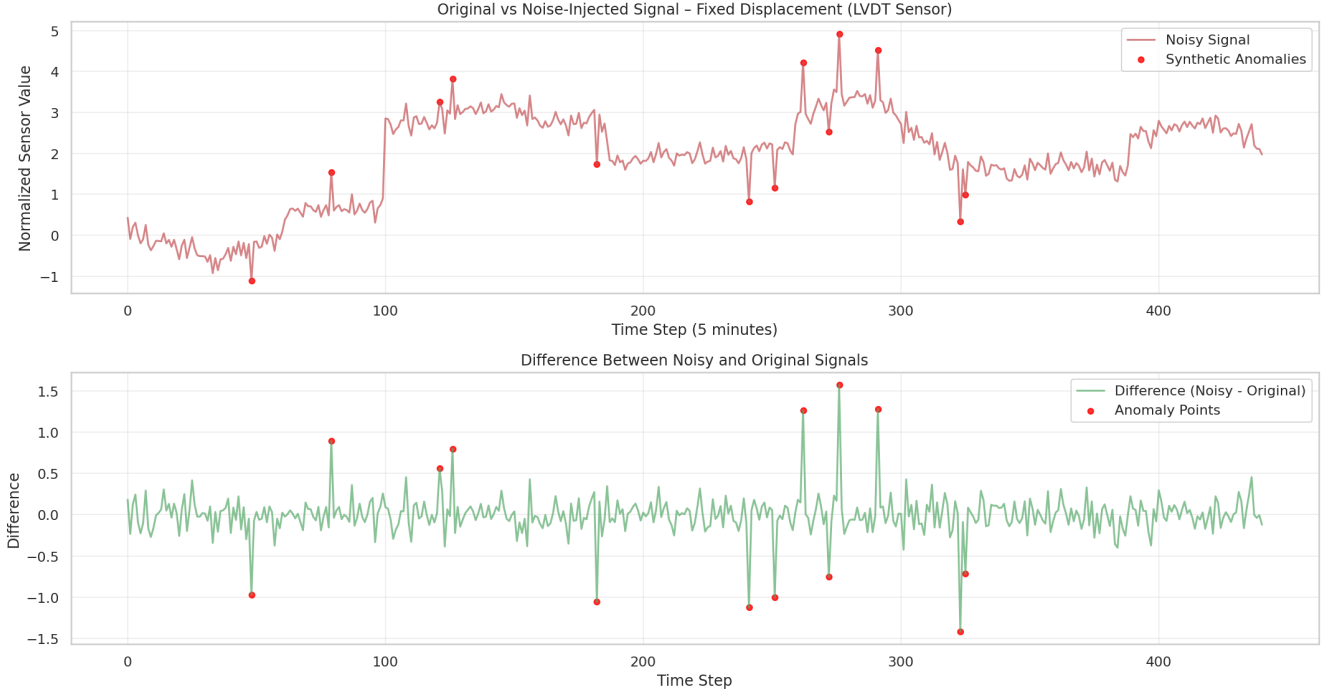


Figure 12: Normalized noise-injected test signal from an LVDT sensor with synthetic anomalies, along with the corresponding difference between the injected noise and the original signal. These differences serve as ground truth labels for model validation. The parameters $\sigma_j^{\max}$ and $\sigma_j^{\min}$ are set to 0.2 and 0.1, respectively.

The ROC curve and corresponding performance metrics are summarized in the plots below (Figure 13). The detection threshold used to generate the ROC curve is selected as the MSE value that maximizes the F1-score. In this case, the parameters are set to a 90% percentile and a significance level of $\alpha = 0.7$. As described in Section 4.5, in real deployments, the choice of these parameters requires a compromise: setting the threshold too high results in very few samples exceeding it, which introduces substantial bias in the estimation; conversely, setting the threshold too low may underestimate abnormal values, leading to inaccuracies in anomaly detection.

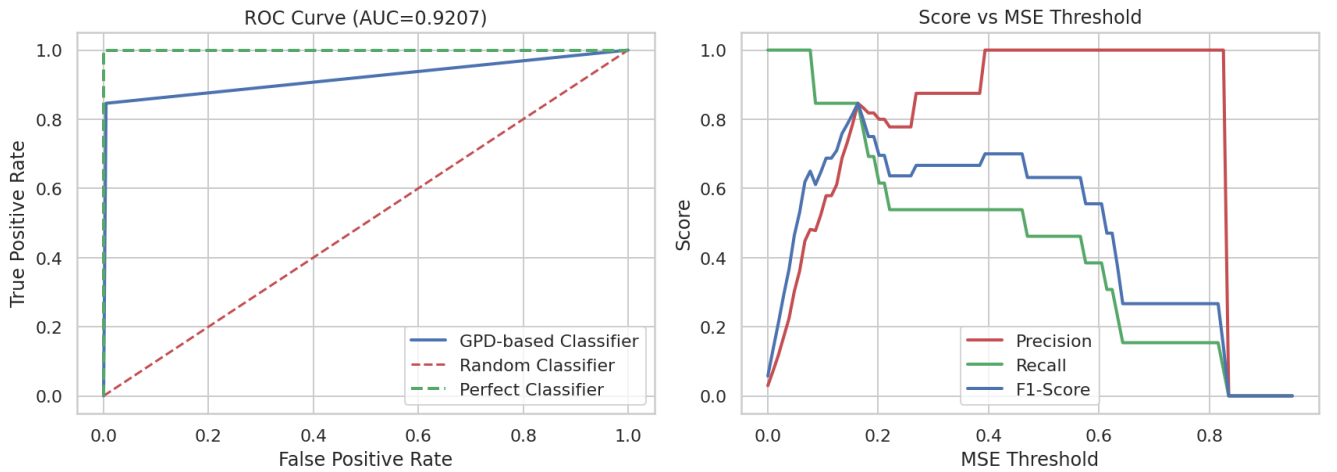


Figure 13: ROC curve illustrating an area under the curve (AUC) of 0.9207, indicating strong anomaly detection performance. Precision, recall, and F1-score are plotted as functions of the MSE threshold, which is determined using extreme value analysis as described in Section 4.4. The maximum F1-score achieved is 0.8462.

Practical Limitations

While the proposed approach has demonstrated effectiveness in detecting anomalies, achieving an AUC of 0.9207 and F1-score of 0.8462 as summarized in Figure 13, several limitations must be acknowledged. First, the magnitude of the injected noise may have been too large for a fully realistic assessment of the model’s performance. Additionally, the limited length of the available data constrains both training and testing phases, potentially affecting robustness. The results are also sensitive to hyperparameter choices (e.g., k , d_{model} , number of layers, epochs), highlighting the need for careful calibration. These findings suggest that further tuning and evaluation on larger and more diverse datasets are necessary before real-world deployment. Overall, the methodology shows promise, but additional research is needed to ensure robustness and optimal parameter selection.

6 Conclusion

This report presents a machine learning-based anomaly detection framework for structural health monitoring (SHM) of a viaduct, developed within the IAM4RAIL project at CEMOSA. The proposed methodology integrates Kalman filtering, power spectral density analysis, a two-layer stacking ensemble for environmental compensation, and a TimesBlock architecture for anomaly detection via Extreme Value Analysis (EVA).

The Kalman filter was implemented to illustrate its potential for mitigating environmental drift in sensor signals and for detecting structural damage in pot bearings. Under the assumption that an optimally designed Kalman filter produces residuals that approximate white noise in both time and frequency domains, any significant deviation—such as increased correlation—could be interpreted as indicative of structural damage. This principle was used to apply a Bayesian whiteness test and to define a damage index. The Kalman filter was successfully implemented and the Bayesian whiteness test performed, which proved the residuals were not truly white. However, the observed correlation in the residuals is likely attributable to suboptimal filter tuning rather than structural damage. Consequently, this limits the reliability of the Bayesian test as a damage indicator in its current form. Future implementations should consider more sophisticated modeling, such as incorporating finite element analysis of the bridge to better represent resonant frequencies and improve filter performance.

The two-layer stacking ensemble method proposed in [9] was successfully implemented to reduce environmental variability. While theoretically robust, it proved overly complex relative to the size of the available dataset. In practice, it occasionally led to overfitting or signal flattening. Although further tuning may improve performance, alternative dimensionality reduction or trend-removal techniques such as Principal Component Analysis (PCA) or Auto-Regressive Integrated Moving Average (ARIMA) might offer more interpretable and scalable solutions. Addressing the environmental effects is still relevant for the anomaly detection step.

The TimesBlock architecture showed strong potential for time-series anomaly detection, particularly by capturing both inter- and intra-periodic patterns in structural behavior. By transforming univariate signals into 2D variation embeddings, the model effectively learned short- and long-term temporal dependencies. Synthetic anomaly injection was used for controlled validation, yielding an Area Under the Curve (AUC) of 0.9207 and a maximum F1-score of 0.8462, indicating promising performance in detecting deviations from normal structural behavior.

Despite these results, several practical limitations remain. The method is sensitive to hyperparameter choices (e.g., number of layers, embedding size, learning rate), and the several assumptions within the EVA framework may not always hold in real-world scenarios. Additionally, the computational cost of the TimesBlock model could be a bottleneck for real-time applications. The reliance on clean, anomaly-free training data can be problematic when such data is not available or cannot be guaranteed.

In summary, the proposed methodology presents a promising approach for improving anomaly detection in SHM systems. Future work should focus on refining the Kalman filter design to enable the practical application of the Bayesian whiteness test, investigating different environmental compensation strategies, and validating the full framework on larger and more diverse datasets towards scalable and deployable SHM solutions.

References

- [1] *Instrumentación y Monitorización en el Viaducto del Barranco del Arroyo de las Huertas de Mateo, Minglanilla (Cuenca): Proyecto e Instalación de la Instrumentación en el Viaducto*. Informe Técnico Informe 1, Tomo Único. Elaborado para el Ministerio de Transportes y Movilidad Sostenible y ADIF. España: Laboratorio Central de Estructuras y Materiales, Ministerio de Transportes y Movilidad Sostenible, ADIF, 2025.
- [2] Kalil Erazo et al. “Vibration-based structural health monitoring under changing environmental conditions using Kalman filtering”. In: *Mechanical Systems and Signal Processing* 139 (2020), p. 106587.
- [3] Ayoub Keshmiry et al. “Effects of Environmental and Operational Conditions on Structural Health Monitoring and Non-Destructive Testing: A Systematic Review”. In: *Sensors* 23.6 (2023), p. 2989. DOI: 10.3390/s23062989.
- [4] Dan Simon. *Optimal State Estimation: Kalman, H Infinity, and Nonlinear Approaches*. John Wiley & Sons, 2006.
- [5] Juntao Kang et al. “Effective alerting for bridge monitoring via a machine learning-based anomaly detection method”. In: *Measurement* 210 (2023), p. 112981. DOI: 10.1016/j.measurement.2022.112981.
- [6] Fernando J. Tenelema, Rick M. Delgadillo, and Joan R. Casas. “Bridge Damage Detection and Quantification under Environmental Effects by Principal Component Analysis”. In: *Proceedings of the Technical University of Catalonia (UPC-BarcelonaTech)* (2020). Available upon request or institutional repository.
- [7] Qian Li et al. “Dynamic Warning Method for Structural Health Monitoring Data Based on ARIMA: Case Study of Hong Kong–Zhuhai–Macao Bridge Immersed Tunnel”. In: *Journal of Bridge Engineering* 26.5 (2021), p. 05021002. DOI: 10.1061/(ASCE)BE.1943-5592.0001705.
- [8] Haixu Wu et al. “TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis”. In: *International Conference on Learning Representations*. 2023.
- [9] Lei Wang et al. “Online diagnosis for bridge monitoring data via a machine learning-based anomaly detection method”. In: *Engineering Structures* 284 (2023), p. 115894. DOI: 10.1016/j.engstruct.2023.115894.
- [10] Chaoli Zhang et al. “TFAD: A Decomposition Time Series Anomaly Detection Architecture with Time-Freq Analysis”. In: *Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM 2022)*. 2022, pp. 2497–2507. DOI: 10.1145/3511808.3557470.
- [11] Arun Raj. *Understanding Inception: Simplifying the Network Architecture*. <https://medium.com/swlh/understanding-inception-simplifying-the-network-architecture-54cd31d38949>. Accessed: 2025-08-01. 2018.
- [12] Diederik P Kingma and Jimmy Lei Ba. “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980* (2015). <https://arxiv.org/abs/1412.6980>.
- [13] Muhali. *Generalized Pareto Distribution - Wikipedia Commons*. https://commons.wikimedia.org/wiki/File:Generalised_Pareto_distribution_PDF.png. Licensed under CC BY-SA 4.0. 2018.
- [14] Neema Davis, Gaurav Raina, and Krishna Jagannathan. “LSTM-based anomaly detection: Detection rules from extreme value theory”. In: *arXiv preprint arXiv:1909.06041* (2019). URL: <https://arxiv.org/abs/1909.06041>.
- [15] Jesse Davis and Mark Goadrich. “The relationship between Precision-Recall and ROC curves”. In: *Proceedings of the 23rd international conference on Machine learning*. ACM. 2006, pp. 233–240. DOI: 10.1145/1143844.1143874.

7 Appendix

7.1 Gamma Weights $\gamma_{j,k}$

As stated in section 3.2 the test set of the meta model $\mathbf{M}$ is calculated as $\mathbf{M} = [\mathbf{M}_1, \dots, \mathbf{M}_j]$ with:

$$\mathbf{M}_j = \frac{1}{K} \sum_{k=1}^K \gamma_{j,k} \cdot F_{j,k}(\mathbf{R}) \quad (41)$$

where $\gamma_{j,k}$ are weights that combine both accuracy and temporal weights. These weights are a linear combination of two different weights $a_{j,k}$ and $b_{j,k}$. Note these weights are defined differently as those mentioned in [5].

1. $a_{j,k}$ favors the meta-models with the lower root mean squared error:

$$a_{j,k} = \frac{e^{-\text{rmse}_{j,k}}}{\sum_{k=1}^K e^{-\text{rmse}_{j,k}}} \quad (42)$$

2. $b_{j,k}$ favors more recent folds by assigning higher importance to later iterations, improving time-dependent model performance:

$$\beta_{j,k} = \frac{(k+1)^{\frac{1}{\lambda}}}{\sum_{k=0}^{K-1} (k+1)^{\frac{1}{\lambda}}} \quad (43)$$

3. The weights $\gamma_{j,k}$ are defined as a linear combination of these weights: $\gamma_{j,k} = c_a \cdot a_{j,k} + c_b \cdot b_{j,k}$ where c_a and c_b are adjustable coefficients. In this implementation, $c_a = c_b = 0.5$.